Projet 5
API à permissions et départ d'un employé

Gestion des Identités et des Accès (IAM) : Rapport d'examen

OAuth 2.0 · OpenID Connect · LDAP

Nom, prénom :	**Amal Loukil**
Date :	Novembre 2026
Durée :	2 heures, travail individuel

Sommaire

1. Contexte et objectif · 2. Architecture · 3. Modèle d'autorisation · 4. Mise en œuvre · 5. Durée des jetons et révocation · 6. Matrice de tests · 7. Protocole de départ · 8. Comparaison des deux modes · 9. Recommandation chiffrée · 10. Limites et conclusion · Annexe : reproduire les mesures

1. Contexte et objectif

Un jeton d'accès JWT est une preuve d'identité qui se vérifie sans interroger personne : l'API contrôle sa signature et sa date d'expiration. Cette force est aussi une faiblesse : **tant que le jeton n'est pas expiré, l'API l'accepte, même si la session qui l'a produit est terminée**. Pour une entreprise, cela pose une question concrète : *que reste-t-il comme accès à un employé après son départ ?*

Ce projet répond en deux temps. D'abord, construire une API protégée avec des permissions fines : des **scopes** attribués au client, des **rôles** issus des groupes de l'annuaire LDAP, et une règle de **propriété** de la ressource. Ensuite, **mesurer** le délai avant que chaque accès tombe quand on retire l'employé de ses groupes puis qu'on désactive son compte, et comparer deux façons de valider un jeton : la validation locale et l'introspection (RFC 7662).

2. Architecture

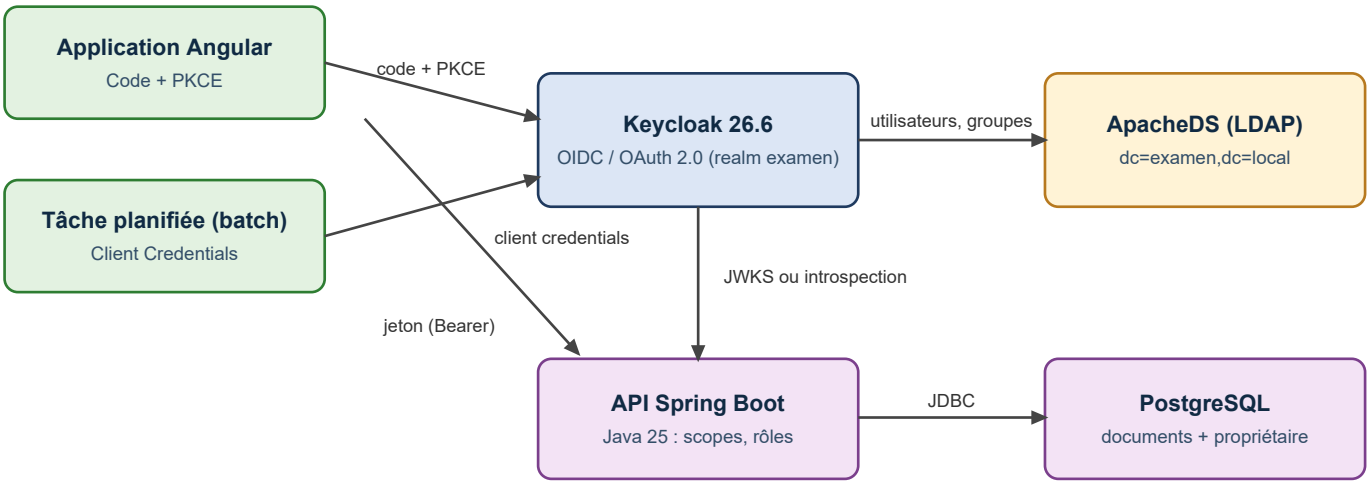


Figure 1 : architecture de la solution (flèches : sens des échanges).

L'annuaire contient les employés et leurs groupes. Keycloak authentifie l'utilisateur en vérifiant son mot de passe dans l'annuaire, lit ses groupes et délivre un jeton. L'application Angular obtient ce jeton par le flux *Authorization Code avec PKCE* ; la tâche planifiée l'obtient par *Client Credentials*. L'API Spring Boot reçoit le jeton dans l'en-tête *Authorization: Bearer*, le valide, applique les droits, puis lit ou écrit les documents dans PostgreSQL.

Composant	Technologie	Rôle
Annuaire	ApacheDS 2.0.0.AM26 (Apache Directory Studio 2.0.0-M17), suffixe dc=examen,dc=local, port 10389	Stocke les employés et les groupes
Serveur d'identité	Keycloak 26.6.0, realm « examen »	Authentifie, délivre les jetons
API	Spring Boot 4.1.1, Java 25 (Amazon Corretto)	Valide les jetons, applique les droits
Base de données	PostgreSQL 16 (port 5433)	Documents et propriétaire
Application web	Angular (dernière version), PKCE écrit à la main	Connexion et appels à l'API
Scripts	Python 3.12	Configuration, tests et mesures

3. Modèle d'autorisation

Une requête est acceptée si **quatre conditions** sont réunies : le jeton est valide, il porte le scope exigé par la route, l'utilisateur a un rôle autorisé, et, pour une suppression, il est propriétaire du document (sauf s'il est manager).

3.1 Groupes LDAP, rôles et scopes

Groupe LDAP	Membres	Rôle dans l'API	Droits
Admins	amartin	manager	Tout lire, écrire et supprimer
Developers	bdupont, cdurand	editor	Lire, écrire, supprimer ses propres documents
Staff	les 5 employés	reader	Lire seulement

Les rôles ne sont pas stockés dans l'API : ils sont **déduits des groupes LDAP** présents dans le jeton (revendication *groups*). Un employé de Developers et de Staff cumule les rôles editor et reader. Les **scopes** décrivent ce que le *client* a le droit de demander : l'application web possède docs:read, docs:write et docs:delete ; la tâche planifiée seulement docs:read.

3.2 Routes protégées

Route	Scope requis	Rôles autorisés	Règle de propriété
GET /docs	docs:read	reader, editor, manager, service	aucune
POST /docs	docs:write	editor, manager	le créateur devient propriétaire
DELETE /docs/{id}	docs:delete	editor, manager	un editor ne supprime que ses documents

401 signifie « je ne sais pas qui vous êtes » (jeton absent, falsifié, expiré ou destiné à une autre API). **403** signifie « je sais qui vous êtes, mais vous n'avez pas le droit » (scope, rôle ou propriété insuffisants).

4. Mise en œuvre

4.1 L'annuaire

Dans Apache Directory Studio, un serveur ApacheDS « Examen » héberge la partition *examen* de suffixe *dc=examen,dc=local*. Les utilisateurs sont dans *ou=People* et les groupes dans *ou=Groups* (classe *groupOfNames*, attribut *member* contenant le DN complet de chaque membre).

```
dc=examen,dc=local
├── ou=People
│   ├── uid=amartin   (Alice Martin, CEO)
│   ├── uid=bdupont   (Bob Dupont, Lead Developer)
│   ├── uid=cdurand   (Clara Durand, Developer)
│   ├── uid=dbernard  (David Bernard, Marketing Manager)
│   └── uid=epetit    (Emma Petit, Support Engineer)
└── ou=Groups
    ├── cn=Admins      member: amartin
    ├── cn=Developers  member: bdupont, cdurand
    └── cn=Staff       member: les cinq employés
```

Une contrainte du schéma LDAP : un *groupOfNames* doit contenir au moins un *member*. On ne peut donc pas retirer le dernier membre d'un groupe, ce qui a guidé le choix de l'employé qui « part » dans le protocole de départ (bdupont, car Developers a deux membres).

4.2 Keycloak

Le realm *examen* contient une fédération LDAP (utilisateurs lus dans *ou=People*, groupes lus dans *ou=Groups* par un mappeur *group-ldap-mapper*) et quatre clients :

Client	Type	Usage	Scopes
webapp	public, Code + PKCE (S256)	Application Angular	docs:read, docs:write, docs:delete
batch	confidentiel, Client Credentials	Tâche planifiée	docs:read seulement
api	confidentiel	Introspection par l'API (RFC 7662)	aucun
other	public	Test de l'audience (sans audience api-examen)	docs:read

Deux mappeurs de jeton sont ajoutés : **groups** (les groupes LDAP de l'utilisateur dans le jeton) et **audience** (valeur *api-examen*, pour que l'API refuse les jetons destinés à un autre service). La fédération est réglée en mode *UNSYNCED* (pour pouvoir désactiver un compte dans Keycloak) et sans cache (voir section 7).

4.3 L'API Spring Boot

À chaque requête, l'API : (1) lit le jeton ; (2) le valide ; (3) vérifie le scope ; (4) convertit les groupes en rôles ; (5) vérifie le rôle puis la propriété. La validation du jeton existe en deux modes, choisis par le paramètre *?mode=* :

- **Validation locale** : signature contrôlée avec la clé publique de Keycloak (JWKS, mise en cache), émetteur (*iss*), audience (*aud* = api-examen) et date d'expiration (*exp*, sans aucune tolérance). Aucun appel réseau par requête.
- **Introspection (RFC 7662)** : l'API demande à Keycloak si le jeton est encore actif. Un appel à Keycloak par requête, mais Keycloak connaît l'état réel du compte et de la session.

4.4 L'application Angular (Authorization Code + PKCE)

Le flux est écrit à la main pour en montrer chaque étape : l'application génère un *code_verifier* aléatoire et son empreinte SHA-256 (*code_challenge*), redirige vers Keycloak avec un paramètre *state* (protection contre les attaques CSRF), puis échange le code reçu contre les jetons en présentant le *code_verifier*. Un code volé en chemin est donc inutilisable. La déconnexion révoque le refresh token (RFC 7009). L'interface affiche le jeton décodé, son compte à rebours et un journal coloré des réponses (200, 401, 403).

5. Durée des jetons et révocation

Le jeton d'accès dure **120 secondes**. Le refresh token fait l'objet d'une **rotation** : chaque renouvellement en émet un nouveau et l'ancien devient inutilisable (*Revoke Refresh Token* actif, réutilisation interdite). La révocation suit la RFC 7009 : on envoie le jeton à */protocol/openid-connect/revoke*.

Action	Endpoint /revoke	Refresh ensuite	Access (local)	Access (introspection)
Révocation du refresh token	endpoint -> 200	refresh -> 400	access local -> 200	access introspection -> 401
Révocation de l'access token	endpoint -> 200	refresh -> 200	access local -> 200	access introspection -> 401

Lecture. Révoquer le refresh token empêche tout renouvellement (le renouvellement répond 400), mais l'ancien jeton d'accès reste accepté en validation locale (200) : l'API ne contacte pas Keycloak. En introspection, le même jeton est refusé (401), car Keycloak sait que sa session est terminée. Révoquer le jeton d'accès lui-même produit le même contraste. **La révocation n'a donc d'effet immédiat que là où l'on interroge Keycloak.**

6. Matrice de tests

Chaque profil est testé sur chaque route, dans les deux modes de validation : **56 résultats sur 56 sont conformes** à ce qui est attendu. Voici le mode local ; le mode introspection donne exactement les mêmes codes.

Profil	GET /docs	POST /docs	DELETE /docs/1 (doc de amartin)	DELETE /docs/2 (doc de bdupont)
amartin (groupe Admins = manager)	200	201	200	200
bdupont (groupe Developers = editor)	200	201	403	200
epetit (groupe Staff = reader)	200	403	403	403
tâche planifiée (client credentials)	200	403	403	403
amartin via le client « other » (mauvaise audience)	401	401	401	401
jeton falsifié	401	401	401	401
sans jeton	401	401	401	401

- **amartin (manager)** peut tout faire, y compris supprimer le document de bdupont.
- **bdupont (editor)** supprime son propre document (200) mais pas celui d'amartin (403) : règle de propriété.
- **epetit (reader)** peut lire (200) mais ni écrire ni supprimer (403) : rôle insuffisant.
- **La tâche planifiée** n'a que docs:read : elle lit (200) et le reste est refusé (403) car le scope manque.
- **Mauvaise audience, jeton falsifié, absence de jeton** : 401, l'identité n'est pas établie.

7. Protocole de départ d'un employé

L'employé qui part est **bdupont** (Bob Dupont, Lead Developer). Le protocole comporte deux étapes, dont on mesure l'effet sur chaque type d'accès :

- **Étape 1 : retirer ses groupes** (Developers et Staff) dans l'annuaire.
- **Étape 2 : désactiver son compte** dans Keycloak.

7.1 Effet du retrait des groupes

Un **nouveau** jeton ne contient plus ses groupes après **2.2 s** ; son POST /docs est refusé (403). L'**ancien** jeton, lui, se comporte différemment selon le mode : en validation locale il porte toujours les groupes (POST /docs : 201), alors qu'en introspection Keycloak recalcule les droits à chaque appel et refuse (403).

7.2 Effet de la désactivation du compte

Accès	Délai après la désactivation	Explication
Nouvelle connexion	2.1 s	Keycloak vérifie l'état du compte à chaque connexion
Refresh token	4.1 s	un compte désactivé ne peut plus renouveler son jeton
Jeton d'accès, introspection	4.2 s	l'API demande à Keycloak, qui répond « inactif »
Jeton d'accès, validation locale	**108.8 s**	l'API ne contacte personne : le jeton vit jusqu'à son expiration



Figure 2 : délai avant que chaque accès de l'employé parti soit refusé.

Constat. Après le départ, la connexion, le renouvellement et l'introspection tombent en quelques secondes. Mais un jeton validé localement reste utilisable pendant **109 secondes**, soit presque toute sa durée de vie de 120 s. Les délais de quelques secondes des trois premières lignes dépendent de la machine de test, peu puissante : seul l'écart d'ordre de grandeur avec la validation locale est significatif.

7.3 Un piège : le cache de la fédération LDAP

Lors des essais avec le réglage par défaut de Keycloak (*cache policy* = *DEFAULT*), le retrait d'un groupe dans l'annuaire n'était plus vu par les nouveaux jetons après un appel d'introspection : Keycloak gardait en cache l'utilisateur et ses groupes. Pour un protocole de départ, c'est dangereux, car l'employé conserve ses rôles. La correction est de régler la fédération sur **NO_CACHE** (les groupes sont relus à chaque jeton), au prix d'une requête LDAP par émission de jeton. La désactivation du compte, elle, invalide ce cache immédiatement. Ce comportement se reproduit avec l'option *--ldap-cache DEFAULT* du script de configuration.

8. Comparaison des deux modes de validation

Mode	Appels à l'API	Appels à Keycloak	Moyenne (ms)	Médiane (ms)	95e centile (ms)
Validation locale	100	0	20.3	17.3	34.4
Introspection	100	100	39.5	35.8	55.8

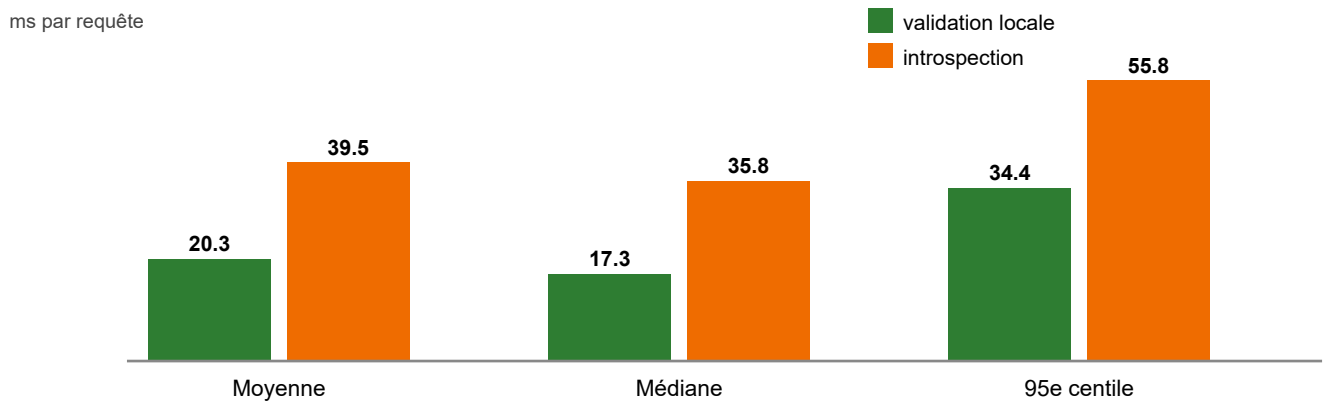


Figure 3 : temps de réponse de l'API pour 100 requêtes, selon le mode de validation.

Critère	Validation locale	Introspection
Délai de révocation	jusqu'à l'expiration du jeton (mesuré : 109 s)	quelques secondes (mesuré : 4.2 s)
Appels à Keycloak	0 par requête (la clé publique est mise en cache)	1 par requête
Temps de réponse moyen	20.3 ms	39.5 ms (+19.2 ms, +95 %)
Dépendance à Keycloak	faible : l'API fonctionne même s'il est indisponible	forte : si Keycloak est en panne, l'API ne peut plus valider

L'introspection coûte environ **19 ms de plus** par requête et fait peser sur Keycloak une charge proportionnelle au trafic de l'API. En échange, elle réduit la fenêtre d'exposition d'un employé parti de 109 s à 4 s environ.

9. Recommandation chiffrée

Le compromis tient en une phrase : **un jeton court réduit l'exposition mais multiplie les renouvellements ; l'introspection supprime presque l'exposition mais coûte un appel par requête**. Avec la validation locale, la fenêtre maximale d'accès d'un employé parti est égale à la durée de vie du jeton.

Niveau de risque	Exemples	Durée du jeton	Mode	Exposition maximale après un départ
Faible	lecture de données publiques	15 minutes	local	15 minutes
Moyen	données internes de l'entreprise	5 minutes	local	5 minutes
Élevé	RH, finance, suppression de données	1 à 2 minutes	introspection (ou local avec jeton très court)	environ 4 secondes (introspection)

Justification chiffrée. (1) Aujourd'hui, avec un jeton de 120 s en validation locale, l'exposition mesurée est de 109 s ; passer à 15 minutes la porterait à 900 s, soit environ 8 fois plus. (2) L'introspection ajoute 19 ms par requête : pour une API qui traite 50 requêtes par seconde, cela représente 50 appels par seconde vers Keycloak, contre un renouvellement tous les 120 s par utilisateur en validation locale. (3) On réserve donc l'introspection aux opérations sensibles, et on garde la validation locale, rapide et indépendante, pour le reste.

10. Limites et conclusion

Limites

- Le flux « Direct Access Grant » du client webapp sert uniquement à automatiser les tests ; en production seul le flux Code + PKCE devrait rester actif.
- Les routes /_reset et /metrics et le paramètre ?mode= de l'API sont réservés à la démonstration et ne doivent jamais être exposés.
- Les comptes de test partagent un mot de passe d'exercice, et le chiffrement LDAPS est désactivé (environnement local).
- Les mesures sont faites sur une seule machine, peu puissante : les délais de quelques secondes sont indicatifs ; l'ordre de grandeur (secondes contre durée de vie du jeton) est, lui, significatif.
- Un seul employé et un seul scénario de départ ont été mesurés.

Conclusion

Le projet montre qu'**authentifier n'est pas révoquer**. Les permissions fines (scopes, rôles issus de l'annuaire, propriété de la ressource) donnent à chaque profil exactement les droits prévus : 56 cas sur 56 sont conformes. Mais la qualité d'un protocole de départ dépend surtout de la façon dont l'API valide les jetons : en validation locale, un employé parti garde son accès pendant presque toute la durée du jeton (109 s mesurées) ; en introspection, l'accès est coupé en quelques secondes (4.2 s), pour un surcoût d'environ 19 ms par requête. Il faut enfin penser aux caches : celui de la fédération LDAP peut masquer le retrait d'un groupe. La bonne configuration associe une durée de jeton proportionnée au risque, l'introspection pour les opérations sensibles, un cache désactivé sur les groupes et la désactivation du compte en dernière étape du départ.

Annexe : exécution dans Docker

Le projet complet (annuaire, base de données, Keycloak, API et application) se lance aussi dans des conteneurs avec **une seule commande** : *docker compose up -d --build* (mode d'emploi dans README-DOCKER.md). Dans ce mode, l'annuaire est **OpenLDAP**, avec les mêmes fichiers LDIF que l'ApacheDS du reste du projet (mêmes employés, mêmes groupes, même suffixe). Les jetons portent *iss* = *http://localhost:8080*, alors que les conteneurs joignent Keycloak par *http://keycloak:8080* : l'API distingue donc l'adresse publique (contrôle de l'émetteur) de l'adresse interne (clé publique et introspection).

Les mêmes tests donnent **56 résultats conformes sur 56**. Mesures du protocole de départ et de la comparaison des modes dans cet environnement :

Mesure (Docker, OpenLDAP)	Valeur
Nouveau jeton sans groupe après le retrait	0.1 s
Nouvelle connexion après désactivation	0.0 s
Refresh token après désactivation	0.0 s
Jeton, introspection	0.1 s
Jeton, **validation locale**	**119.1 s**
Temps de réponse moyen, validation locale	14.7 ms
Temps de réponse moyen, introspection	41.9 ms

Même constat qu'avec ApacheDS : la validation locale garde l'accès presque toute la durée de vie du jeton, l'introspection le coupe presque immédiatement (les écarts de quelques secondes dépendent de la machine).

Annexe : reproduire les mesures

Ordre de lancement : serveur ApacheDS dans Studio (*Run*), Keycloak, configuration de Keycloak, API, application Angular. Puis, dans un terminal avec l'environnement Python activé :

```
python apps/scripts/preparer_ldap.py       # charge les 10 entrées dans l'annuaire
python apps/scripts/setup_keycloak.py      # configure Keycloak (realm, clients, scopes, LDAP)
python apps/scripts/tests_matrix.py        # matrice 200/201/401/403 (56 cas)
python apps/scripts/bench.py               # validation locale contre introspection
python apps/scripts/revocation.py          # révocation RFC 7009
python apps/scripts/depart.py              # protocole de départ, délais mesurés
python apps/scripts/generer_rapport_pdf.py # régénère ce document avec les derniers chiffres
```

Comptes de test (mot de passe *secret*) : **amartin** (manager), **bdupont** (editor), **epetit** (reader). Console Keycloak : http://localhost:8080 (admin / admin). Application : http://localhost:4200.